

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**УЕБ СИСТЕМА ЗА НАБЛЮДЕНИЕ НА
ПОКАЗАТЕЛИ В РЕАЛНО ВРЕМЕ: НТТР И
WEBSOCKET СЪРВЪР НА C++ И КЛИЕНТ БЕЗ
ПРИЛОЖНА РАМКА**

КУРСОВ ПРОЕКТ

по дисциплина „Програмиране за Интернет“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Огнян Наков

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД .	3
1.1. Развитие на динамичното уеб съдържание	3
1.2. Изместени технологии и защо са изместени	3
1.3. Двупосочен обмен между браузър и сървър	4
1.4. Формати за структуриран обмен	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Протокол за поток от данни	5
2.2. Приложна рамка на клиента	5
2.3. Съответствие между технологиите от учебната програма и избраните . . .	6
2.4. Формат на обмена	6
2.5. Език и среда на сървъра	7
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	8
3.1. Обща структура	8
3.2. Договор между сървъра и клиента	8
3.3. Свързване на данни на клиента	9
3.4. Проектни ограничения	9
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	10
4.1. Организация на изходния текст	10
4.2. Мрежов слой на сървъра	10
4.3. Сериализатори	10
4.4. Клиентска част	11
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	12
5.1. Какво се проверява	12
5.2. Измервани величини и начин на измерване	12
5.3. Условия за валидност на измерването	12
5.4. Апаратна и програмна среда	13

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	14
6.1. Закъснение по двата канала	14
6.2. Сравнение на двата формата за обмен	14
6.3. Поведение при нарастващ брой клиенти	14
6.4. Обобщение на измерените резултати	15
ЗАКЛЮЧЕНИЕ	16
ЛИТЕРАТУРА	17
ПРИЛОЖЕНИЯ	18

УВОД

Наблюдението на работеща система е задача, при която закъснението има цена. Ако показателят стигне до оператора със закъснение от няколко секунди, картината на екрана описва състояние, което вече не съществува. Класическият уеб модел, при който браузърът пита сървъра през равни интервали и получава пълна страница в отговор, е зле пригоден за тази задача: той харчи заявки, когато нищо не се е променило, и въпреки това изостава, когато промяната настъпи между две запитвания. Двупосочната връзка, при която сървърът изпраща стойност в момента, в който я узнае, решава и двата проблема наведнъж.

Настоящият курсов проект разработва **Pulse**: уеб система за наблюдение на показатели в реално време. Сървърната част е програма на C++20, която сама обслужва HTTP/1.1 [1] за първоначалното зареждане на приложението и WebSocket [2] за потока от стойности след това. Клиентската част е страница без приложна рамка, която стъпва само на браузърните стандарти: обектния модел на документа [3], модела на събитията, структурната графика във формат SVG [4] и разположението с CSS Grid [5]. Отсъствието на рамка е проектно решение, а не пропуск, и е обосновано в Раздел II.

Целта на проекта е да се изгради работеща система за наблюдение в реално време, при която всяка тема от учебното съдържание на дисциплината е представена от действащ елемент на системата, а не от отделен демонстрационен пример. За постигането на целта са формулирани следните **задачи**:

1. преглед на технологиите за обмен на данни между браузър и сървър и на тяхното историческо развитие, включително на изместените от практиката решения;
2. анализ на възможните проектни решения (протокол за поток, формат на данните, начин на изобразяване, наличие или отсъствие на приложна рамка) и обосновка на направения избор;
3. проектиране на архитектурата на системата и на договора между сървъра и клиента;
4. програмна реализация на сървъра на C++20 и на клиента на JavaScript без рамка;
5. изграждане на методика за експериментална проверка на закъснението и на цената на двата формата за обмен;
6. провеждане на измерванията и анализ на получените резултати.

Обхватът на работата е един процес на сървър, който събира показатели и ги раздава, и една страница в браузър, която ги изобразява. Извън обхвата остават разпределеното съхранение на исторически редове, удостоверяването на потребители и разгръщането в производствена среда.

Структура на изложението. Раздел I разглежда предметната област и развитието на технологиите за динамични уеб приложения. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката на експеримента, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Развитие на динамичното уеб съдържание

Първото поколение уеб приложения генерира целия документ на сървъра и го изпраща наново при всяко действие на потребителя. Динамиката е на ниво страница: състоянието живее в сървъра, браузърът е програма за изобразяване. Технологиите от този период, сред които Active Server Pages и генерирането на разметка чрез шаблони от страна на сървъра, решават задачата за променливо съдържание, но не и задачата за променливо съдържание *без презареждане*. Всяка промяна струва един пълен обмен и едно ново построяване на документа.

Второто поколение измества част от състоянието в браузъра. Скриптовият език на клиента [6] получава достъп до построеното дърво на документа и може да го променя след като страницата е заредена. Спецификацията на този достъп днес е стандартизирана като обектен модел на документа [3], а разметката и вградените в нея интерфейси като жив стандарт [7]. Едновременно с това се появява асинхронната заявка от скрипт, която по-късно се формализира в общ модел за извличане [8]. От този момент нататък страницата може да поиска данни, без да се разрушава.

1.2. Изместени технологии и защо са изместени

Част от терминологията в учебното съдържание на дисциплината идва от периода, в който разширяването на браузъра ставаше чрез двоични компоненти. Технологията ActiveX позволява изпълнението на native код в контекста на страницата и е налична само в един браузър на една операционна система. Тя решава реален проблем за времето си, защото тогавашният браузър няма нито графика извън растерни изображения, нито мрежов достъп извън заявката за документ. Изместена е по три причини: обвързаност с една платформа, модел на сигурността, който дава на страницата правата на потребителя, и постепенното покриване на същите нужди от самите стандарти. Аналогично, филтрите и преходите, реализирани като собствено разширение на един производител, днес са част от каскадните стилови спецификации и работят във всеки браузър.

Този проект третира изброените технологии *исторически*. Всяка от темите на

дисциплината е реализирана със стандартизирания си наследник, а съответствието между двете е представено в Раздел II. **[TODO: списък на конкретните теми от учебната програма с точното им позоваване, след сверка със силабуса на дисциплината]**

1.3. Двупосочен обмен между браузър и сървър

Периодичното запитване (*polling*) е най-простият начин да се получат нови данни: с всяка заявка клиентът пита дали има промяна. Цената е компромис между излишни заявки при кратък интервал и закъснение при дълъг. Задържаното запитване (*long polling*) намалява едното за сметка на сложност в сървъра, но остава в рамките на модела „заявка, отговор“ на HTTP [9].

Двата съвременни механизма за поток от сървъра към клиента са събитията, изпращани от сървъра (част от разметковия стандарт [7]), и протоколът WebSocket [2]. Първият остава в рамките на HTTP и е еднопосочен. Вторият е отделен протокол, който започва като HTTP заявка за надграждане и след успешно ръкостискане превръща вече установената връзка в двупосочен канал за съобщения. Сравнението между двата е основната проектна алтернатива на този проект и е разгледано в Раздел II.

1.4. Формати за структуриран обмен

За обмена на структурирани данни в мрежата съществуват две широко приети представяния. Разширяемият език за разметка [10] носи схема, пространства от имена и утвърден инструментариум за преобразуване и проверка. Нотацията за обекти на JavaScript [11] е по-кратка, съответства пряко на структурите от данни на клиентския език и се разчита от браузъра без допълнителна библиотека. Проектът реализира *и двата* формата за един и същ поток от показатели, за да може разликата между тях да бъде измерена, а не описана по памет. **[TODO: показателите, по които двата формата ще бъдат сравнени: размер на съобщението, време за разбор, обем на кода за обработка]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Протокол за поток от данни

Разгледани са четири варианта за доставяне на нови стойности до браузъра: периодично запитване, задържано запитване, събития, изпращани от сървъра, и WebSocket [2]. Първите два остават в модела „заявка, отговор“ и плащат за всяка стойност цената на пълен обмен от заглавни части. Третият премахва тази цена, но е еднопосочен: клиентът няма как да смени наблюдавания набор от показатели по същия канал и му е нужна отделна заявка за всяко управляващо действие.

Избран е WebSocket, защото задачата изисква и двете посоки: сървърът изпраща стойности, клиентът изпраща команди за смяна на набора, за смяна на честотата и за смяна на формата на обмена. Събитията, изпращани от сървъра, остават реализирани като втори, резервен канал, което позволява двата подхода да бъдат измерени един срещу друг върху един и същ поток от данни, вместо да бъдат сравнявани по литературни твърдения. **[TODO: окончателно решение дали резервният канал влиза в предадената версия]**

2.2. Приложна рамка на клиента

Съвременната практика решава задачата за свързване на данни с приложна рамка, която сама следи кои части от документа зависят от кои стойности. Разгледани са два пътя: рамка от познатите днес семейства и клиент без рамка, който работи направо срещу обектния модел на документа [3].

Избран е клиентът без рамка. Мотивът е учебен и е съществен за оценяването: цялото съдържание на дисциплината, което се отнася до клиентската част (обектният модел, колекциите, събитията, свързването на данни, структурната графика), при използване на рамка се превръща в извикване на нейни функции и остава скрито. Без рамка същите теми са видими в изходния текст на проекта. Цената на това решение е ръчно написан слой за свързване на данни, чието проектиране е описано в Раздел III.

2.3. Съответствие между технологиите от учебната програма и избраните

Темите от учебната програма, които сочат към технологии, изместени от практиката, са покрити от стандартизираните им наследници. Съответствието е дадено в таблицата по-долу (Табл. 1), а обосновката за всеки ред следва логиката от Раздел I: избраната технология покрива същата нужда, но е част от стандарт и работи във всеки съвременен браузър.

Таблица 1. Съответствие между технологиите от учебната програма и реализираните в проекта

Тема	Реализирано с	Основание
Двоични компоненти в страницата (ActiveX)	стандартни браузърни интерфейси	еднаква работа във всеки браузър, без права на потребителя за страницата
Генериране на страници на сървъра (ASP)	генериране на HTTP отговор от сървъра на C++20	същата роля, без обвързаност с една платформа
Филтри и преходи като собствено разширение	каскадни стилови преходи и филтри	стандартизирани и хардуерно ускорени
Растерна графика за диаграми	структурна графика SVG [4]	всеки елемент е възел в документа и приема събития
Свързване на данни чрез компонент	собствен слой за свързване върху обектния модел [3]	темата остава видима в изходния текст

2.4. Формат на обмена

Двата разгледани формата, разширяемият език за разметка [10] и нотацията за обекти на JavaScript [11], не се изключват взаимно. Проектът реализира и двата зад един и същ договор за съобщение и позволява формата да се превключва по време на работа. Това решение превръща избора между тях от предположение в измерване и осигурява материала за Раздел VI.

2.5. Език и среда на сървъра

Сървърът е на C++20 [12]. Езикът е избран заради прякото управление на паметта и предвидимото време за реакция, което е съществено, когато измерваната величина е закъснението на самата система. Разгледана е и възможността да се използва готов сървър на приложения, при която обработката на протокола отпада; тя е отхвърлена, защото обработката на HTTP/1.1 [1] и на ръкостискането по WebSocket [2] е част от учебното съдържание, а не спомагателна работа.

Като *предходна работа на същия автор* съществува библиотека за уеб приложения на C++ с планировчик на входа и изхода, изграден върху съпрограми. Проектът може да я ползва като външна зависимост, но не съдържа неин изходен текст: Pulse е самостоятелно хранилище. **[TODO: окончателно решение дали библиотеката влиза като зависимост, или сървърът стъпва само на стандартната библиотека]**

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Обща структура

Системата се състои от два процеса и един канал между тях. Сървърният процес съдържа четири части: източник на показатели, който произвежда стойности през определен интервал; регистър на абонаментите, който помни кой клиент кои показатели наблюдава; сериализатор, който превръща една стойност в съобщение в избрания формат; и мрежов слой, който говори HTTP/1.1 [1] при първоначалното зареждане и WebSocket [2] след надграждането на връзката. Клиентският процес е страницата в браузъра и съдържа приемник на съобщения, хранилище на състоянието, слой за свързване на данни и изобразяващ слой.

Разделянето на сериализатора от източника на показатели е носещото проектно решение. То позволява един и същ поток от стойности да бъде излъчен в разширяемия език за разметка [10] или в нотацията за обекти [11], без нито един ред от източника или от регистъра да се променя. Без това разделяне сравнението между двата формата не би било честно, защото щеше да сравнява две различни реализации, а не два формата.

[TODO: диаграма на общата структура, с легенда за разчитане на означенията]

3.2. Договор между сървъра и клиента

Обменът се описва с малък брой видове съобщения. От клиента към сървъра: заявка за абонамент за списък от показатели, отказ от абонамент, смяна на интервала и смяна на формата. От сървъра към клиента: потвърждение на абонамента, пакет със стойности и съобщение за грешка. Всяко съобщение носи вид, пореден номер и полезен товар.

Поредният номер съществува по една причина: без него клиентът няма как да отличи изгубено съобщение от съобщение, което просто още не е дошло. Тъй като WebSocket доставя съобщенията в реда на изпращането им по вече установена връзка, поредният номер служи за откриване на прекъсване и за повторно установяване на състоянието след ново свързване, а не за пренареждане. **[TODO: пълна таблица на видовете съобщения с полетата им и с примери в двата формата]**

3.3. Свързване на данни на клиента

Свързването на данни е реализирано като явен списък от двойки: път до стойност в състоянието и функция, която прилага тази стойност върху възел от документа. При пристигане на пакет със стойности се обхождат само двойките, чиито пътища са засегнати от пакета. Този подход е избран вместо пълно преизчисляване на изгледа, защото при честота на обновяване от порядъка на кадрите в секунда пълното преизчисляване прави повече работа, отколкото има променени стойности.

Изобразяващият слой е разделен на две. Числовите показатели се изписват като текстови възли и се обновяват през свойството за текстово съдържание. Диаграмите се изграждат като структурна графика [4]: всяка точка от реда е възел в документа, така че върху нея може да се закачи обработчик на събитие и да се приложи стилив преход. Разположението на съставните части на екрана е зададено с CSS Grid [5].

3.4. Проектни ограничения

Проектът приема три ограничения, които стесняват обхвата и трябва да бъдат заявени, преди да са измерени резултати. Първо, състоянието на сървъра се пази в паметта на процеса, което означава, че историята се губи при рестарт. Второ, поддържа се един източник на показатели, а не обединяване на няколко източника. Трето, няма удостоверяване на потребители: всеки клиент, който достигне до крайната точка, получава пълния поток. **[TODO: проверка дали ограничението за удостоверяване е приемливо за условията на заданието]**

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Организация на изходния текст

Хранилището разделя изходния текст на три части. Заглавните файлове с публичните договори стоят в `include/`, реализацията в `src/`, а клиентската част в `web/`. Тестовите са в `tests/`. Изграждането се управлява от CMake, което позволява проектът да се компилира с различни компилатори без промяна в описанието на изграждането. **[TODO: дърво на хранилището с кратко описание на всеки файл, след като реализацията приключи]**

4.2. Мрежов слой на сървъра

Мрежовият слой обработва два протокола върху един и същ слушащ сокет. Първата заявка по всяка връзка е обикновена HTTP/1.1 заявка [1]. Ако тя носи заглавните части за надграждане към WebSocket, слойът изчислява отговора на ръкостискането по правилото от спецификацията [2] и предава връзката на кадровия анализатор. В противен случай заявката се обслужва като заявка за статичен ресурс и връзката се затваря или се запазва според семантиката на HTTP [9].

Кадровият анализатор на WebSocket е мястото, където проектът е най-чувствителен към грешки, защото работи с данни, дошли от мрежата. Дължината на полезния товар се чете от кадъра, тоест от ненадежден източник, и затова се проверява срещу горна граница, преди да бъде заделена памет. Кадрите от клиент към сървър задължително носят маска, а кадър без маска се отхвърля вместо да се обработва. **[TODO: листинг на анализатора на кадри с коментар на изходния текст]**

4.3. Сериализатори

Двата сериализатора реализират един и същ вътрешен договор: приемат пакет със стойности и връщат низ за изпращане. Реализацията за нотацията за обекти [11] се грижи за екранирането на специалните знаци в низовете. Реализацията за разширяемия език за разметка [10] се грижи за екранирането на знаците със специално значение в разметката и за коректното изписване на празни елементи. И двете се проверяват със самостоятелни тестове, защото грешка в екранирането не се вижда в обичайния случай и се проявява

само при определени входни данни.

4.4. Клиентска част

Клиентската част е един документ, един стилев файл и няколко скриптови модула без външни зависимости. Модулът за връзката отговаря за установяването на канала и за повторното свързване след прекъсване. Модулът за състоянието прилага пристигналите пакети върху текущата картина. Модулът за свързване на данни изпълнява описания в Раздел III списък от двойки. Модулът за диаграмите изгражда и обновява структурната графика [4].

Обработката на събития е съсредоточена на едно място чрез делегиране: обработчикът се закача на общ родителски възел и разпознава източника на събитието по неговите данни, а не по отделен обработчик на всеки възел. Това решение произтича пряко от факта, че възлите на диаграмата се създават и унищожават при всяко обновяване.

[TODO: листинги на клиентските модули и екранни снимки на готовия интерфейс]

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Какво се проверява

Проверяват се три твърдения, които проектните решения от Раздел II приемат за верни. Първо, че двупосочният канал доставя стойност до екрана по-бързо от периодичното запитване при еднаква честота на обновяване. Второ, че двата формата за обмен се различават по обем на съобщението и по време за разбор на клиента. Трето, че системата запазва работоспособност при нарастващ брой едновременни клиенти до определена граница, която експериментът трябва да намери, а не да предположи.

Всяко от трите твърдения е записано тук *преди* провеждането на измерването. Целта е една погрешна прогноза да остане видима и да бъде отчетена като погрешна, вместо да бъде пренаписана след получаването на резултата.

5.2. Измервани величини и начин на измерване

Закъснението се измерва от момента, в който източникът произведе стойност, до момента, в който клиентът приложи тази стойност върху документа. За целта всяко съобщение носи времеви отпечатък от сървъра, а клиентът записва своя отпечатък при прилагането. Двата процеса се изпълняват на една машина, за да отпадне въпросът за синхронизация на часовниците. **[TODO: описание на конкретния източник на време на всяка от двете страни и на неговата разделителна способност]**

Обемът на съобщението се измерва като брой байтове на кадъра по мрежата за един и същ пакет със стойности, сериализиран в двата формата. Времето за разбор се измерва на клиента около извикването на разбора. Пропускателната способност се измерва като брой доставени пакети за единица време при нарастващ брой едновременни връзки. **[TODO: инструментът, с който се създава натоварването, и начинът, по който се установява, че самият инструмент не е тясното място]**

5.3. Условия за валидност на измерването

Едно измерване се приема за валидно само ако са изпълнени следните условия. Машината не изпълнява друго забележимо натоварване по време на опита. Всеки опит се повтаря няколко пъти, а резултатът се отчита с мярка за разсейване, а не само със средна

стойност. Първите измервания след стартиране на процеса се отхвърлят, защото включват еднократни разходи по подготовката. Броят на повторенията и продължителността на всеки опит се фиксират предварително и не се променят след като са видени резултати. **[TODO: конкретни стойности за брой повторения, продължителност на опит и брой отхвърлени начални измервания]**

5.4. Апаратна и програмна среда

Средата на измерването е част от резултата и се описва изцяло, за да може опитът да бъде повторен. **[TODO: процесор, обем на паметта, операционна система и нейната версия, компилатор и ниво на оптимизация, браузър и неговата версия]**

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Закъснение по двата канала

Измерването сравнява закъснението от произвеждането на стойността до нейното изобразяване при доставка по WebSocket [2] и при периодично запитване, при еднаква честота на обновяване. Формата на резултата е дадена в таблицата по-долу (Табл. 2). Отчитат се средна стойност и мярка за разсейване по условията от Раздел V.

Таблица 2. Закъснение от произвеждане до изобразяване по вид на канала

Канал	Средно	Разсейване	Брой опити
WebSocket	[TODO:]	[TODO:]	[TODO:]
Периодично запитване	[TODO:]	[TODO:]	[TODO:]

[TODO: коментар на резултата: потвърждава ли се прогнозата от Раздел V, и ако не се потвърждава, каква е установената причина]

6.2. Сравнение на двата формата за обмен

За един и същ пакет със стойности се измерват обемът на съобщението по мрежата и времето за неговия разбор на клиента при сериализация в разширяемия език за разметка [10] и в нотацията за обекти [11]. Формата на резултата е в таблицата по-долу (Табл. 3).

Таблица 3. Обем и време за разбор по формат на обмена

Формат	Обем	Време за разбор	Брой опити
XML	[TODO:]	[TODO:]	[TODO:]
JSON	[TODO:]	[TODO:]	[TODO:]

[TODO: коментар: доколко разликата в обема се запазва след компресия на транспортно ниво, и има ли случай, в който по-обемният формат е за предпочитане]

6.3. Поведение при нарастващ брой клиенти

Измерва се броят доставени пакети за единица време и закъснението при нарастващ брой едновременни връзки, докато системата престане да поддържа зададената честота на

обновяване. Границата е резултат от измерването, а не проектна цел. **[TODO: графика с брой връзки по хоризонталната ос и доставени пакети по вертикалната, с легенда за разчитане на показателите]**

6.4. Обобщение на измерените резултати

[TODO: обобщение: кои от трите твърдения от Раздел V се потвърждават, кои се опровергават, и какви са установените ограничения на самото измерване]

ЗАКЛЮЧЕНИЕ

В рамките на курсовия проект е разработена система за наблюдение на показатели в реално време, изградена от сървър на C++20 [12], който сам обслужва HTTP/1.1 [1] и WebSocket [2], и от клиент в брауъра без приложна рамка. Клиентската част стъпва пряко върху обектния модел на документа [3], модела на събитията, структурната графика [4] и разположението с CSS Grid [5], а обменът е реализиран в два формата [10, 11], за да може разликата между тях да бъде измерена.

Предимства на решението. Отсъствието на приложна рамка прави всяка стъпка от обработката видима в изходния текст, а сървър, който сам обработва протокола, дава пълен контрол над момента на изпращане, което е предпоставка за измерването на закъснението. Разделянето на сериализатора от източника на показатели позволява смяна на формата по време на работа, без промяна в останалата част от системата.

Недостатъци и ограничения. Слой за свързване на данни е написан ръчно и изисква явно описание на всяка зависимост, което при разрастване на интерфейса става източник на пропуски. Състоянието се пази в паметта на процеса и се губи при рестарт. Няма удостоверяване на потребители. Тези ограничения са заявени в Раздел III преди измерването, а не открити след него.

Инженерна трактовка на резултатите. [TODO: извод от измерените стойности: какво означават те за избора между двата канала и между двата формата в задача от този вид]

Насоки за по-нататъшна работа. Естествените продължения са съхранение на исторически редове извън паметта на процеса, обединяване на няколко източника на показатели, удостоверяване на потребители и разширяване на слоя за свързване на данни с автоматично проследяване на зависимостите.

ЛИТЕРАТУРА

- [1] R. Fielding, M. Nottingham, and J. Reschke, “HTTP/1.1,” Request for Comments RFC 9112, Internet Engineering Task Force, June 2022.
- [2] I. Fette and A. Melnikov, “The WebSocket protocol,” Request for Comments RFC 6455, Internet Engineering Task Force, Dec. 2011.
- [3] WHATWG, “DOM standard.” Living Standard. **[TODO: дата на достъп]**.
- [4] W3C SVG Working Group, “Scalable vector graphics (SVG) 2.” W3C Specification. **[TODO: статус на документа и дата на достъп]**.
- [5] W3C CSS Working Group, “CSS grid layout module level 1.” W3C Specification. **[TODO: статус на документа и дата на достъп]**.
- [6] Ecma International, “ECMA-262: ECMAScript language specification.” Standard. **[TODO: издание и година на цитираното издание]**.
- [7] WHATWG, “HTML standard.” Living Standard. **[TODO: дата на достъп]**.
- [8] WHATWG, “Fetch standard.” Living Standard. **[TODO: дата на достъп]**.
- [9] R. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” Request for Comments RFC 9110, Internet Engineering Task Force, June 2022.
- [10] W3C XML Core Working Group, “Extensible markup language (XML) 1.0 (fifth edition).” W3C Recommendation, Nov. 2008.
- [11] T. Bray, “The JavaScript object notation (JSON) data interchange format,” Request for Comments RFC 8259, Internet Engineering Task Force, Dec. 2017.
- [12] ISO/IEC, “ISO/IEC 14882:2020, programming languages, C++.” International Standard, 2020.

ПРИЛОЖЕНИЯ

Приложение А. Формат на съобщенията

Тук се дава пълното описание на видовете съобщения от договора между сървъра и клиента, описан в Раздел III, заедно с по един пример за всяко съобщение в двата формата на обмена [10, 11]. **[TODO: таблица с полетата на всяко съобщение и примерите към нея]**

Приложение Б. Избрани части от изходния текст

Тук се включват анализаторът на кадри по WebSocket [2], двата сериализатора и слойта за свързване на данни на клиента, с коментар на изходния текст. **[TODO: листинги, добавени с `lstinputlisting` след като реализацията приключи]**

Приложение В. Екранни форми

Тук се включват екранните снимки на готовия интерфейс: изгледът с числовите показатели, изгледът с диаграмите и състоянието при прекъсната връзка. **[TODO: екранни снимки с надписи под тях]**

Приложение Г. Отчет от измерванията

Тук се включва суровият отчет от измерванията, върху който стъпват таблиците в Раздел VI, заедно с описанието на средата от Раздел V. **[TODO: отчет от измерванията]**